

TP n° 5 : Transactions et contrôle de concurrence

Remarque: En cas de difficulté à vous connecter rappelez-vous à la section préparation de l'environnement de travail de la série n° 1.

Pour ce TP, je vous conseille d'utiliser l'interface en ligne de commande *sqlplus* pour vous connecter à vos comptes.

Vous déposez le travail réalisé sur l'ENT.

Exercice n° 1 Atomicité d'une transaction

Commencer par ouvrir deux sessions (S_1 et S_2) et exécuter la commande suivantes dans chacune des sessions (dans la suite de cet exercice, et à chaque connexion (ouverture de session), commencé par exécuter cette commande) :

```
1 SET AUTOCOMMIT OFF
```

1. Créer la table *transaction* dont le script est donné ci-dessous, en utilisant la session S_1 .

```
1 CREATE TABLE transaction (  
2 idTransaction VARCHAR2 (44) , valTransaction NUMBER  
3 (10));
```

2. En utilisant la session S_2 , insérer quelques lignes, modifier une ligne, en supprimer une autre et enfin annuler les mises à jour venant d'être effectuées avec la commande *ROLLBACK*. Afficher le contenu de la table.
3. Insérer à nouveau dans *transaction* quelques lignes et clore avec la commande *quit*. Consulter le contenu de la table, en utilisant la session S_1 . Que s'est-il passé ?
4. Dans la session S_1 , insérer quelques lignes et fermer *brutalement sqlplus*. Reconnecter à nouveau sur votre compte. Les données saisies ont-elles été préservées ?
5. Dans une nouvelle session, insérer quelques lignes et modifier la structure de la table *transaction*, en y ajoutant par exemple l'attribut *val2 transaction* de type *NUMBER(10)*. Exécuter la commande *ROLLBACK*. Que s'est-il passé ?
6. Conclure : définir de manière précise qu'est-ce qu'une session, une transaction et comment valider une transaction ou l'annuler.

Exercice n° 2 Transactions concurrentes

Créer le schéma de la base de données de gestion des billets d'avions, constituée des deux relations suivantes¹ :
Client(idClient, prenomClient, placesReserveesClient) et
Vol (idVol, capaciteVol, placesReserveesVol)

```
1 CREATE TABLE vol(  
2 id Vol VARCHAR2 (44) ,  
3 capacite Vol NUMBER (10) ,  
4 nbrPlacesReserveesVol NUMBER (10)  
5 );
```

```
1 CREATE TABLE client(  
2 id Client VARCHAR2 (44) , prenom Client  
3 VARCHAR2 (11) ,  
4 nbrPlacesReserveesCleint NUMBER (10)  
5 );
```

¹ Il s'agit d'un schéma simplifié mais suffisant, pour étudier les mécanismes transactionnels

Executer deux transactions T_1 et T_2 en parallèle (donc dans deux sessions différentes). Contrairement à l'exercice précédent où on a systématiquement utilisé *SET AUTOCOMMIT OFF*, avant le début de chaque session, dans cet exercice, on laisse le réglage d'isolation par défaut. Si vous utilisez ORACLE, ce réglage est *READ COMMITTED*. Et que certaines versions ne permettent pas d'obtenir les niveaux d'isolation *REPEATABLE READ* et *UNCOMMITTED READ*.

Insérer un vol et deux clients.

```
1 CREATE TABLE client{
2 id Client VARCHAR 2 (44) , prenom Client
3 VARCHAR 2 (11) ,
4 nbrPlacesReserveesCleint NUMBER (10)
5 };
```

Isolation des transactions

Effectuer une réservation pour un client et ne validez pas encore cette transaction (T_1). Vous devriez constater que :

- les mises à jour sont visibles par la transaction qui les a effectuées (T_1)²,
- elles sont en revanche invisibles par toute autre transaction (ici T_2).

C'est dû à la notion d'isolation des transactions. L'isolation est complète quand le résultat d'une transaction ne dépend pas de la présence ou non d'autres transactions.

COMMIT et ROLLBACK

Effectuez un *ROLLBACK* de la transaction en cours (T_1). Vous devriez constater :

- que la base revient à son état initial,
- qu'exécuter un **COMMIT** ou un **ROLLBACK** n'ont plus aucun effet, car nous sommes au début d'une nouvelle transaction (une session peut générer plusieurs transactions mais exécutée en série, voir le cours).
- Pour la transaction T_2 tout se passe comme si T_1 n'avait pas existé.

Recommencez la transaction T_1 et validez, cette fois, les mises à jour (*COMMIT*). Vous devriez constater que :

- il n'est pas possible d'annuler par un **ROLLBACK**. Une fois les données validées, elles sont pérennes. Le fait de ne pas pouvoir annuler correspond à la notion de durabilité. Autrement dit, un commit valide les données de manière définitive. Après une validation, les données intègrent l'état de la base (nous appelons 'état de la base à un instant t ' l'ensemble des données validées).

Maintenant, la transaction T_2 voit les mises à jour de T_1 , et on en déduit que le niveau d'isolation par défaut d'Oracle est celui de *READ COMMITTED*. Par contre dans le cas où le niveau d'isolation est *REPEATABLE READ*³, T_2 ne verra pas ces mises à jour, car elle a commencé avant T_1 , et continue à accéder à l'état de la base au moment elle a commencé son exécution.

²Ceci permet d'assurer la cohérence des données

³les mêmes lectures renvoient toujours le même résultat

isolation incomplete = incoherence possible

Réinitialisez, toujours en mode *READ COMMITTED*. Maintenant déroulez deux transactions T_1 (réservation de 2 billets pour c_1) et T_2 (réservation de 3 billets pour c_2) en parallèle selon l'alternance suivante :

- on effectue les deux sélections pour T_1 (vol et client) ;
- on effectue les deux sélections pour T_2 (vol et client) ;
- on effectue les deux mises à jour de T_1 , et on valide ;
- on effectue les deux mises à jour de T_2 , et on valide

L'objectif de cette manipulation est de reproduire le problème des mises à jour perdues, dans le cas où le niveau d'isolation n'est pas maximal. Une fois les deux transactions terminées, vous devriez constater que les deux clients ont bien réservé $2+3 = 5$ billets, mais que seulement 3 billets ont été réservés pour le vol.

La base se retrouve dans un état incohérent. Chaque transaction, vue individuellement, est correcte. Il y a donc un défaut de concurrence dû à un niveau d'isolation incomplet.

isolation complète = blocage et rejet des transactions possibles

Pour assurer une cohérence complète, il faut choisir le mode d'isolation complet (serializable). Réinitialisez les deux sessions, en choisissant ce mode, et refaites la même exécution concurrente précédente. Cette fois, une des deux transactions sera rejetée.

Reproduire l'exemple suivant d'une exécution concurrente, vu en cours :

$$r_1(d)w_2(d)w_2(d')C_2w_1(d')C_1$$

Conclure : l'algorithme implémenté dans ORACLE est-il le même que celui vu en cours (verrouillage à deux phases) ?.

Exercice 1: Atomicité d'une transaction

Question 1 : Créer la table et insérer des données

Réponse :

La table a été créée avec succès et des données ont été insérées. Les opérations d'insertion sont visibles dans la session courante.

Question 2 : Insérer, modifier, supprimer puis exécuter ROLLBACK

Réponse :

Après avoir effectué des insertions, une mise à jour et une suppression, l'exécution de la commande ROLLBACK annule toutes les modifications non validées.

La table revient à son état initial.

Cela montre que la transaction est **atomique** : soit toutes les opérations sont validées, soit aucune ne l'est.

Question 3 : Insérer des données puis quitter sans COMMIT

Réponse :

Si la session est fermée sans exécuter COMMIT, les modifications ne sont pas sauvegardées. Lors de la reconnexion, les données insérées ne sont plus présentes.

Les modifications non validées sont automatiquement annulées.

Question 4 : Fermeture brutale de la session

Réponse :

En cas de fermeture brutale, les modifications non validées sont perdues. La base de données revient à l'état avant la transaction.

Question 5 : ALTER TABLE puis ROLLBACK

Réponse :

Après une commande ALTER TABLE, même si on exécute ROLLBACK, la modification de structure reste.

Les commandes DDL (comme ALTER TABLE) provoquent un **commit implicite** ou ne sont pas annulées comme les opérations DML.

Question 6 : Définir session, transaction, COMMIT et ROLLBACK

Réponse :

- Une **session** est une connexion entre un utilisateur et le SGBD.
- Une **transaction** est un ensemble d'opérations exécutées comme une unité logique.
- COMMIT valide définitivement les modifications.
- ROLLBACK annule toutes les modifications non validées.

Exercice 2: Transactions concurrentes

Question 1 : Tester l'isolation entre deux transactions

Réponse :

Une transaction voit ses propres modifications, mais les autres transactions ne les voient pas tant qu'il n'y a pas de COMMIT. Cela montre l'**isolation** des transactions.

Question 2 : Effet de ROLLBACK

Réponse :

Après un ROLLBACK, toutes les modifications de la transaction sont annulées. Les autres transactions ne voient aucun changement. La base revient à l'état initial.

Question 3 : Effet de COMMIT

Réponse :

Après un COMMIT, les modifications deviennent permanentes et visibles par toutes les autres transactions.

Un ROLLBACK après un COMMIT n'a aucun effet.

Question 4 : Résultat des deux transactions (tes tests)

Réponse :

Dans notre exécution :

- Client C1 a réservé 2 places
- Client C2 a réservé 3 places
- Le vol contient 5 places réservées

La base est **cohérente**, car : $2 + 3 = 5$

Question 5 : Expliquer les mises à jour perdues

Réponse :

Une mise à jour perdue se produit lorsque deux transactions lisent les mêmes données initiales, puis les modifient sans tenir compte des modifications de l'autre.

Exemple :

- T1 lit 0, écrit 2
- T2 lit 0, écrit 3

Résultat final :

- Clients = 5 places
- Vol = 3 places

La base devient incohérente.

Question 6 : Mode SERIALIZABLE

Réponse :

Avec le niveau d'isolation SERIALIZABLE, le SGBD empêche les conflits entre transactions. Une des transactions peut être bloquée ou rejetée. Cela évite les anomalies comme les mises à jour perdues.

Question 7 : Oracle utilise-t-il exactement le verrouillage à deux phases ?

Réponse :

Non. Oracle n'utilise pas exactement le verrouillage strict à deux phases vu en cours. Il utilise un mécanisme de contrôle de concurrence basé sur la lecture cohérente (MVCC). Son comportement est donc différent du modèle théorique simple.

```
mysql> SELECT * FROM transaction_tp5;
```

idTransaction	valTransaction	remarque
T4	400	NULL
T5	500	NULL

```
2 rows in set (0.000 sec)
```

```
mysql> SELECT * FROM vol_tp5;
```

idVol	capaciteVol	nbrPlacesReserveesVol
V1	200	0

```
1 row in set (0.001 sec)
```

```
mysql> SELECT * FROM client_tp5;
```

idClient	prenomClient	nbrPlacesReserveesClient
C1	Ali	0
C2	Sara	0

idTransaction	valTransaction	remarque
T4	400	NULL
T5	500	NULL
T4	400	NULL
T5	500	NULL

4 rows in set (0.001 sec)

Query OK, 0 rows affected (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

idTransaction	valTransaction	remarque
T4	400	NULL
T5	500	NULL
T4	400	NULL
T5	500	NULL

idVol	capaciteVol	nbrPlacesReserveesVol
V1	200	2

1 row in set (0.000 sec)

Query OK, 0 rows affected (0.000 sec)

idClient	prenomClient	nbrPlacesReserveesClient
C1	Ali	2
C2	Sara	0

2 rows in set (0.000 sec)

idVol	capaciteVol	nbrPlacesReserveesVol
V1	200	2

1 row in set (0.000 sec)